

Google Internship Prep — Detailed Subtopic Roadmap

Order: DSA (ongoing) → Git/GitHub → Python → Concurrency → Networking → Distributed Systems → SQL → REST APIs → Chatbot Project → NumPy/Pandas → NLP → LLM Integration → Docker

1. DSA (ongoing throughout — never "finish" this)

- **Arrays & Strings** — two pointers, sliding window, prefix sums, in-place modification
 - **Linked Lists** — singly/doubly, reversal, cycle detection (Floyd's), merge operations
 - **Stacks & Queues** — monotonic stack, circular queue, implementing queue with stacks
 - **Trees** — binary trees, BST insert/delete/search, traversals (inorder/preorder/postorder/level-order), height & balance checks, tries
 - **Heaps / Priority Queues** — heapify, k-th largest/smallest, merge k sorted lists, top-k problems
 - **Graphs** — BFS, DFS, topological sort, Dijkstra, Bellman-Ford, Union-Find (DSU), MST (Kruskal/Prim)
 - **Hashing** — hash maps/sets, collision handling, design problems (LRU cache, design HashMap)
 - **Sorting & Searching** — merge sort, quicksort, binary search + its variants (search in rotated array, etc.)
 - **Recursion & Backtracking** — permutations, combinations, subsets, N-Queens, Sudoku solver
 - **Dynamic Programming** — 1D/2D DP, knapsack variants, LCS, LIS, DP on grids/trees
 - **Greedy algorithms** — interval scheduling, activity selection
 - **Bit manipulation** — XOR tricks, bitmasking for subsets
 - **Complexity analysis** — Big-O/Theta/Omega, amortized analysis
 - Practice via a structured pattern list (e.g. NeetCode 150 / Blind 75) rather than random problems
-

2. Git + GitHub

- Core commands — `init`, `add`, `commit`, `status`, `log`, `diff`
- Branching — create, switch, delete, naming conventions
- Merging — fast-forward vs merge commits
- **Merge conflicts** — resolving manually, understanding conflict markers
- **Rebasing** — interactive rebase, when to rebase vs merge
- Remotes — `push`, `pull`, `fetch`, `clone`, tracking branches

- GitHub workflow — forking, pull requests, code review etiquette
 - `.gitignore`, stashing changes
 - Tags & releases
 - GitHub Actions basics — simple CI workflow (run tests on push)
 - Commit message conventions (conventional commits)
-

3. Python Fundamentals

- Core syntax, data types, control flow, comprehensions
 - Functions — `*args`/`**kwargs`, default args, lambdas, closures, decorators
 - OOP in Python — classes, inheritance, magic methods (`__init__`, `__repr__`, etc.), dataclasses
 - Iterators & generators — `yield`, generator expressions
 - Exception handling — `try/except/finally`, custom exceptions
 - File I/O, context managers (`with` statement)
 - Standard library — `collections` (deque, Counter, defaultdict), `itertools`, `functools`
 - **`asyncio` basics** — event loop, coroutines (`async`/`await`), tasks (critical for chatbot later)
 - Virtual environments, `pip`, `requirements.txt`
 - Type hints (`typing` module)
-

4. Concurrency & Multithreading

- Recap: process vs thread (you have OS background — connect it here)
 - **The GIL** — what it is, why Python threads don't give true CPU parallelism
 - `threading` module — `Thread`, `Lock`, `RLock`
 - Synchronization primitives — mutexes, semaphores, condition variables
 - **Producer-consumer pattern** — implement with `queue.Queue`
 - Thread pools — `ThreadPoolExecutor`, `ProcessPoolExecutor`
 - Race conditions — how they arise, how to spot them in code
 - Deadlock, livelock, starvation — causes and prevention strategies
 - `multiprocessing` vs `threading` — when to use which (CPU-bound vs I/O-bound)
 - How `asyncio` concurrency differs from threading (single-threaded cooperative multitasking)
 - Thread-safe data structures
-

5. Networking

- OSI model & TCP/IP stack (high-level mapping)
 - **TCP vs UDP** — reliability, ordering, use cases
 - HTTP/HTTPS — request/response cycle, methods, status codes, headers
 - DNS resolution — how a domain becomes an IP
 - Basic socket programming (Python `socket` module) — build a tiny client/server
 - **WebSockets** — handshake process, persistent connection model, message framing (direct chatbot prerequisite)
 - REST vs gRPC — brief conceptual comparison
 - TLS/SSL basics — what "secure" actually means in HTTPS
 - Load balancers — brief intro (ties into distributed systems next)
-

6. Distributed Systems Basics

- **CAP theorem** — consistency, availability, partition tolerance trade-offs
 - Consistency models — strong vs eventual consistency
 - Replication — leader-follower, multi-leader replication
 - Sharding / partitioning strategies — range-based, hash-based
 - Load balancing strategies — round robin, least connections, consistent hashing
 - Message queues — pub-sub pattern, conceptual Kafka/RabbitMQ understanding
 - Fault tolerance — retries, idempotency, timeouts, circuit breakers
 - Consensus algorithms — know Raft/Paxos exist and what problem they solve (no deep math needed)
 - Distributed caching — brief concept (cache invalidation, TTLs)
-

7. SQL

- Core queries — `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`
 - **Joins** — inner, left, right, full outer, self-join
 - **Indexing** — B-tree indexes, when they help vs hurt write performance
 - **Normalization** — 1NF through 3NF, and when denormalization is the right call
 - Transactions & **ACID** properties
 - Isolation levels (brief — read committed, serializable, etc.)
 - Query optimization basics, reading an `EXPLAIN` plan
 - Relational vs NoSQL — concrete criteria for choosing each
 - SQLAlchemy ORM basics (direct chatbot prerequisite)
-

8. REST APIs

8. REST APIs

- ☐ REST principles — statelessness, resource-based URLs
 - ☐ HTTP methods mapped to CRUD operations
 - ☐ Status codes — what each range means and when to use which
 - ☐ Authentication — API keys, **JWT**, OAuth basics
 - ☐ API design — versioning, pagination, consistent error responses
 - ☐ Building endpoints with FastAPI (or Flask)
 - ☐ API documentation — OpenAPI/Swagger basics
-

9. Chatbot Backend Project (apply everything above)

- ☐ Architecture design — client/server split, where WebSocket vs REST is used
 - ☐ FastAPI project structure
 - ☐ WebSocket endpoint — handle connect/disconnect/broadcast
 - ☐ Concurrent connection handling using `asyncio`
 - ☐ JWT auth integration on REST endpoints
 - ☐ Database schema — Users, Conversations, Messages tables
 - ☐ SQLAlchemy integration with the schema
 - ☐ Apply **Observer pattern** (broadcasting messages) and **Strategy pattern** (swappable LLM providers)
 - ☐ Basic logging and error handling
 - ☐ Unit tests for message-handling logic
 - ☐ Write a short "how I'd scale this to 1M users" design doc (sharding, load balancing WebSockets, retry logic for LLM calls)
-

10. NumPy + Pandas

- ☐ NumPy — arrays, vectorization, broadcasting, slicing/indexing
 - ☐ Pandas — Series vs DataFrame, indexing, filtering
 - ☐ Data cleaning — missing values, duplicates, type conversion
 - ☐ `groupby`, aggregation, merging/joining DataFrames
 - ☐ Basic plotting (matplotlib/seaborn) — optional but useful for explaining your resume analyzer
-

11. NLP Basics

- ☐ **Tokenization** — word-level vs subword (BPE) — directly relevant to LLM context windows
- ☐ Embeddings — what word2vec captures, contextual embeddings at a conceptual level

- Embeddings — what word2vec captures, contextual embeddings at a conceptual level
 - Text preprocessing — stopwords, stemming vs lemmatization
 - Bag of words, TF-IDF (classic baseline techniques)
 - spaCy or Hugging Face basics — load a model, run inference
 - Context window limits — how token count constrains LLM input/output
-

12. LLM API Integration

- API authentication (API keys, environment variable management — never hardcode keys)
 - Prompt construction — system/user/assistant role structure
 - **Streaming responses** — handling token-by-token output instead of blocking calls
 - Token usage & context window management
 - Error handling & rate limit handling (retries with backoff)
 - Function/tool calling concept (how LLMs can invoke external functions)
 - Model selection & basic cost/latency trade-offs
-

13. Docker

- Images vs containers — the conceptual difference
 - Writing a Dockerfile — `FROM`, `COPY`, `RUN`, `CMD`
 - Building and running containers locally
 - **Docker Compose** — multi-container setup (app + database together)
 - Volumes — persisting data outside the container
 - Container networking — how services talk to each other
 - Environment variables / secrets management
 - Basic image optimization — multi-stage builds, smaller base images
-

How to use this

- Check off subtopics as you genuinely understand them (can explain out loud, not just "read about").
- Topic 1 (DSA) runs forever in the background — don't wait to "finish" it.
- Topic 9 (Chatbot) is where 5-8 line items live, but it's really the integration test for everything before it — if a subtopic above feels shaky, you'll find out while building.